Case Study: RUDP, a Reliable Transport over UDP with a Learned Congestion Controller

Course: Computer Networks, VIT Vellore **Faculty:** Anuradha G **Team:** Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819), Shloak Sinha (24BDS0378) **Code:** github.com/Capsturrrr/rudp **Due:** 21 October 2026

1. Summary

UDP delivers packets but promises nothing: packets can be lost, duplicated or reordered. TCP adds reliability and congestion control, but it lives inside the operating system kernel and cannot be changed by an application. We built RUDP, a reliable and congestion-controlled transport in C on top of UDP, then asked whether a small neural network could choose the sending window better than the classic AIMD rule. Against a well-tuned AIMD baseline the neural agent was 15% to 52% faster on four of five test paths and tied on the fifth. It was not better in every condition, and this study reports where it was not.

2. The problem

In October 1986 the early Internet suffered its first congestion collapse: on one path throughput fell from 32 kilobits per second to 40 bits per second, because senders kept retransmitting into an already overloaded network. Jacobson and Karels introduced slow start and congestion avoidance in response. Our problem had two parts:

1. Can TCP's core guarantees (complete, ordered delivery and a self-limiting sending rate) be rebuilt in user space over UDP, and verified?
2. Can a learned policy for the congestion window beat the fixed AIMD rule, measured fairly against a tuned baseline?

User-space transports matter in practice: QUIC, used for much of the web, is a reliable transport built on UDP.

3. What we built

Protocol. Every packet has a 13-byte header (sequence number, acknowledgment number, flags SYN/ACK/FIN/DATA, a CRC-16 checksum and a payload length) with up to 512 bytes of payload. A connection opens with a three-way handshake. The sender uses a sliding window, retransmits by timeout and by fast retransmit (three duplicate ACKs), and uses an adaptive timer in the style of RFC 6298. We first built Go-Back-N, then made Selective Repeat the default: the receiver buffers up to 256 out-of-order packets and every ACK carries a 32-byte bitmap of what it holds, so only lost packets are resent.

Congestion control. AIMD (slow start, then one packet per round trip, halving on loss) and CUBIC are baselines. Two learned controllers choose, about once per round trip, whether to shrink, hold or grow the window: a Q-learning table and a small neural network (5 inputs, two hidden layers of 32, 5 outputs, 1,413 weights) trained as a Deep Q-Network. In the final C sender

the neural agent uses a window cap of 128, a slow-start phase, pacing, and a safety net that hands control to AIMD when 10% of an interval is lost.

Application and tools. A terminal and a web chat run over RUDP, plus file transfer with an end-to-end hash check. We wrote a Wireshark dissector in Lua, a four-router OSPF network in Cisco Packet Tracer (two VLANs, DHCP, an access list, tested failover), a user-space impairment proxy (delay, jitter, loss, a shared bottleneck, cross traffic), and a virtual-time build of the same sender that runs experiments about 1000 times faster than real time.

4. Method

All comparisons use the same transport and timers, so only the window policy differs. Baselines are tuned (AIMD and CUBIC with the same window cap of 128), runs are paired by random seed, and intervals are 95% confidence intervals. The five named paths are 5G-like, lossy (2% loss), satellite-like, fast with high delay, and hostile (10% loss). To avoid fitting to those, we also drew 100 random paths from wide ranges, with separate tuning and test sets. An ablation removed one feature at a time.

5. Results

Completion time in seconds, real clock, neural agent against tuned baselines:

Path	AIMD tuned	CUBIC tuned	Neural	Neural vs AIMD
5G-like	1.09	1.19	0.92	15% faster
Lossy, 2% loss	7.05	13.92	3.62	49% faster
Satellite-like	11.36	8.26	5.46	52% faster
Fast, high delay	8.20	6.52	3.92	52% faster
Hostile, 10% loss	8.34	12.32	8.93	no significant difference

- The same ordering held on real UDP sockets through the impairment proxy.
- On 100 random held-out paths the agent needed 0.53 times the time of tuned AIMD, was faster on 89, and was never more than 20% slower.
- The ablation showed the larger window cap and pacing do most of the work; with the agent's original settings it was only about 14% faster than tuned AIMD.
- Files of 2 MB and 4 MB over impaired paths arrived byte-identical in every run; the 4 MB file took 8.9 s with the agent and 34.6 s with the original capped AIMD.
- **On the real Linux kernel (tc/netem), against tuned AIMD:** the agent was 61% faster on a lossy path, 70% on a 5G-like path and 83% on a satellite path (5 runs each, all highly significant), and 33% faster with reordering (not significant). **Kernel TCP was faster than the agent on every path, by 1.5 to 3.2 times.** We do not claim to beat TCP.
- **Tests for known research gaps (virtual time, 20 paired seeds):** when bandwidth or delay changes during a transfer the agent was fastest in all four cases (6% to 53% faster

than tuned AIMD) at the price of 1% to 6% retransmissions. A simplified BBR-style baseline beat tuned AIMD on four of five paths and was **faster than the agent on the 2% lossy path** (8.2 s against 9.8 s); the agent won on the satellite-like and fast paths. Two agents sharing a link with unequal round-trip times were fairer than two AIMD flows (Jain index 0.84 against 0.64 at +150 ms), though still biased to the short-RTT flow.

- **How far can the network be trusted?** A depth-5 decision tree agrees with the network on only 79% of decisions yet matches its transfer time on four of five paths; noise of 0.2 on its inputs costs at most 12%; but formal interval-bound certification succeeds for only 43% of decisions at a tiny perturbation (0.001) and none at 0.03.
- A fuzzer fed one million random corrupted packets to the parser with none accepted. Our first checksum, a byte sum, let about 2% through, which is why we replaced it with CRC-16.

6. Problems and how we solved them

- **A server that stopped accepting clients** after one was killed mid-transfer: it now accepts a new SYN in any state, and a lost final handshake ACK is recovered when the first data packet arrives.
- **An inflated result.** Early margins over AIMD were larger than the true ones, because a timer defect hurt the baseline more. We found it, fixed it, re-measured everything and corrected the report.
- **Our first kernel runs were wrong.** The netem queue limit also counts packets in flight, so large windows overflowed it on the satellite path; the baseline was untuned; and TCP sent larger payloads. We fixed the setup, re-ran, and kept the failed runs in the record.
- **Fairness training failed.** Training the agent against a simulated competing flow left its share at 59% to 60%, and the simulated competitor did not reproduce real two-flow behaviour, so fairness remains a hand-made rule.
- **Testing artifacts:** a leftover network emulation rule that made a healthy run look hung, Wireshark on Windows not seeing the WSL loopback, and buffered output that made the chat look like it lost messages.
- **Real sockets exposed bugs the emulator hid:** a 512-byte payload overflowing a buffer and spurious timeouts from a too-tight timer.

7. Limits

- **Fairness.** Sharing a link with tuned AIMD, the agent takes about 70% of it. An optional polite mode brings this to about 52%, but it costs speed (the average time over random paths rises from 0.53 to 0.58 of AIMD's, with one path 2.95 times slower), so it is off by default and is a hand-made rule, not learned.
- **Competing traffic.** With bursty cross traffic the agent is 15% faster than tuned AIMD but level with CUBIC, and retransmits about three times as often as AIMD.
- **No gain at 10% random loss**, where the agent hands over to AIMD.
- **Limited network testing.** One kernel netem run on one laptop (5 runs per cell) and a user-space proxy; no multi-machine path, and no measured test of sharing a link with kernel TCP (the script is written; it needs a Linux host with tc). The BBR-style baseline is a simplification, not the Linux BBR. Run-to-run variation is large.

- **Not production grade:** no encryption or multiplexing, 512-byte payloads, 16 clients per server. The sender is also available as a C library (librudp) and as a small web application, RUDP Drop, which races the policies on a chosen network.

8. What we learned

Reliable transport is mostly timers and edge cases. Congestion control is a feedback loop in which the sender listens to the network. A learned policy can help when a path is lossy or long, but its win shrinks against a fair baseline, so the baseline must be tuned before any claim is made. Independent evidence (a packet capture, a hash of the received file, a fuzzer) caught problems our own logs did not. Reporting a smaller, correct margin was better than defending a larger, wrong one.

9. Future work

Closing the gap to kernel TCP, a TCP coexistence test and multi-machine tests; a training environment with a competing flow so fairness is learned; training on recorded traces and cross traffic; a learned hand-over rule; a comparison against the real Linux BBR; learning with formal certificates; encryption and multiplexed streams.

References

1. V. Jacobson and M. Karels, “Congestion Avoidance and Control,” ACM SIGCOMM, 1988.
2. D.-M. Chiu and R. Jain, “Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks,” Computer Networks and ISDN Systems, 1989.
3. M. Allman, V. Paxson, E. Blanton, “TCP Congestion Control,” RFC 5681, 2009.
4. V. Paxson et al., “Computing TCP’s Retransmission Timer,” RFC 6298, 2011.
5. S. Ha, I. Rhee, L. Xu, “CUBIC: A New TCP-Friendly High-Speed TCP Variant,” ACM SIGOPS Operating Systems Review, 2008.
6. A. Langley et al., “The QUIC Transport Protocol: Design and Internet-Scale Deployment,” ACM SIGCOMM, 2017.
7. V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, 2015.